

Code Layout, Readability & Reusability

OptiCrop — Smart Agricultural Production Optimization Engine

Repository Structure

- `app.py` — Flask application: routes for Home, About, Find Your Crop, and the prediction handler.
- `notebooks/EDA_and_Model_Training.ipynb` — full ML pipeline: EDA, pre-processing, model training, evaluation, and artifact saving.
- `model/` — serialized artifacts (`model.pkl`, `scaler.pkl`, `label_encoder.pkl`) used by `app.py` at runtime.
- `templates/` — HTML pages (`home.html`, `about.html`, `find_your_crop.html`), kept separate from logic per Flask convention.
- `static/` — CSS styling, separated from markup.
- `data/` — source dataset, kept separate from code so the pipeline is fully reproducible.

Readability Practices

- Consistent, descriptive variable names (`X_train`, `y_pred`, `label_encoder`) instead of single-letter abbreviations.
- Each notebook section is preceded by a markdown heading describing its purpose (EDA, pre-processing, model training).
- Flask routes are short and single-purpose — one route per page/action, no logic mixed across routes.

Reusability Practices

- The same `StandardScaler` and `LabelEncoder` fitted during training are serialized and reused at inference time, guaranteeing consistent transformations between training and serving.
- Model comparison logic loops over a dictionary of candidate models rather than duplicating training/evaluation code per algorithm.
- The prediction logic (`scale -> predict -> decode`) is isolated in one place in `app.py`, making it easy to swap in a different model file without touching the rest of the application.